

PYTHON MINI PROJECT

SMART EXPENSE TRACKER DOCUMENTATION

Problem Statement

Many individuals struggle to track daily expenses and understand spending patterns. Build a Python application that allows users to log, categorize, and analyze their expenses.

Objectives

- Record daily expenses (date, category, amount, description)
- Categorize spending (Food, Travel, Bills, etc.)
- Generate monthly summaries and insights

Key Features

- CLI or simple GUI input system
- Data storage using CSV or JSON
- Monthly expense summary
- Category-wise breakdown (pie chart using libraries like `matplotlib`)
- Detect highest spending category

Expected Outcome

A tool that helps users understand where their money goes and suggests areas to reduce spending.

1. Project Overview

The Smart Expense Tracker is a Python-based command-line application designed to help users record, manage, analyze and visualize their daily personal expenses. Built using core Python libraries and `matplotlib`, it provides a complete personal finance management experience from adding and editing expenses to generating charts and exporting reports. All data is persisted locally using JSON files, making the application lightweight and portable with no database setup required. The system supports dynamic category management, monthly analytics, visual trend analysis, and export to both CSV and PDF formats.

2. Objectives and Key Concepts Covered

Objectives:

- Build a full-featured expense management CLI application in Python
- Persist data locally using JSON without any external database
- Provide analytical summaries and visual charts for spending insights
- Support export of expense data to CSV and PDF formats
- Allow flexible category management by the user

Key Concepts Covered:

Concept	Description
OOP (Classes & Methods)	All logic encapsulated in the ExpenseTracker class
File I/O	JSON read/write for persistent data storage
Data Validation	Date format, amount, and category validation on every input
Collections & defaultdict	Efficient category-wise aggregation
Matplotlib	Pie chart, bar chart, and line trend visualization
CSV Export	Writing structured expense data using the csv module
PDF Export	Formatted report generation using reportlab
CLI Menu System	Interactive loop-based user interface
Error Handling	Try/except for input validation and file operations

4. Class

Class - ExpenseTracker

The entire application is encapsulated in a single class. On initialization, it resolves the download directory, loads categories from categories.json, and loads existing expense records from expenses.json.

Class Constants:

Constant	Purpose
DATA_FILE	Filename for storing expense records (expenses.json)
CATEGORIES_FILE	Filename for storing categories (categories.json)
DEFAULT_CATEGORIES	Predefined list: Food, Travel, Bills, Entertainment, Shopping, Health, Education, Other

5. Methods

5.1 Initialization & File Management

- **__init__(self)** Initializes the tracker. Sets up the download directory, loads categories, and loads expense data. Acts as the application bootstrap.
- **_resolve_download_dir(self)** Attempts to use a preferred hardcoded export path. If unavailable (e.g., on another machine), it falls back to creating a Downloads/ folder next to the script. Ensures exports always work regardless of environment.
- **_download_path(self, filename)** A utility helper that returns the full path to a file inside the resolved download directory. Used by all export and chart-save functions to maintain consistent output paths.
- **load_data(self) / save_data(self)** Reads and writes the expenses.json file. On load, prints the total number of records or a "no previous data" message. Handles JSONDecodeError gracefully to prevent crashes on corrupt files.
- **load_categories(self) / save_categories(self)** Reads and writes categories.json. If the file does not exist, populates from DEFAULT_CATEGORIES and saves immediately, ensuring categories are always available on first run.

5.2 Category Management

- **display_categories(self)** Prints a numbered list of all available categories. Called before add/search operations so the user can see valid options.

- **add_category(self, name)** Validates that the name is non-empty and not a duplicate, then appends it to the list and saves. Names are automatically converted to title case for consistency.
- **update_category(self, index)** Renames a category by index. Simultaneously updates all existing expense records that reference the old category name, ensuring data consistency across the JSON store.
- **delete_category(self, index)** Deletes a category only if no existing expenses reference it. If linked expenses exist, it warns the user and blocks deletion, preventing orphaned expense records.

5.3 Core Expense Operations

- **add_expense(self, date_str, category, amount, description)**

Validates all four inputs before saving:

- Date must match YYYY-MM-DD format (validated using `datetime.strptime`)
- Category must exist in the current categories list
- Amount must be a positive float
- Description defaults to "No description" if left blank

On success, appends the expense dictionary to `self.expenses` and saves to JSON.

- **view_expenses(self, month=None, year=None)**

Displays all expenses in a formatted table. Accepts optional month and year parameters to filter by a specific month. Results are sorted by date before display. Prints a running total at the bottom.

- **update_expense(self, index)**

Shows current values and prompts for new ones. Pressing Enter for any field keeps the existing value. Validates new inputs the same way as `add_expense`. Updates only the specific record at the given index and saves.

- **delete_expense(self, index)**

Removes the expense at the given index using `list.pop()` and saves. Prints a confirmation showing the deleted record's details.

5.4 Search Functions

- **search_by_category(self, category)** Performs a case-insensitive filter on all expenses matching the given category. Displays date, amount, and description for each matching record.
- **search_by_amount(self, min_amt, max_amt)** Filters expenses where amount falls within the given range (inclusive). Useful for identifying large or small transactions.

5.5 Analytics

monthly_summary(self, month, year)

Provides a comprehensive text-based summary for the selected month:

- Total amount spent
- Number of transactions
- Number of distinct spending days
- Average spend per day
- Highest and lowest spending categories
- Per-category breakdown with percentage and an ASCII progress bar

Also includes a smart note - if any category exceeds 40% of total spending, it flags it as a concern. Otherwise confirms balanced spending.

5.6 Visualization

visualize_monthly(self, month, year)

Generates a side-by-side chart (1 row × 2 columns figure):

- **Left - Pie Chart:** Shows percentage distribution of spending across categories with autopct labels
- **Right - Bar Chart:** Shows absolute amount per category with value labels on top of each bar

Saved as `monthly_YYYY_MM.png` in the downloads folder and displayed on screen.

visualize_daily_trend(self, month, year)

Generates a line chart with area fill showing daily spending totals across the selected month. Each data point is labeled with the amount. Dates are displayed on the X-axis (rotated 45°). Saved as `daily_YYYY_MM.png`.

5.7 Export Functions

export_csv(self)

Uses Python's csv.DictWriter to write all expense records to expenses.csv.

Columns: date, category, amount, description. Handles missing fields gracefully using .get() with defaults.

export_pdf(self)

Uses the reportlab library to generate a styled PDF report:

- Title: "Expense Report"
- Table with columns: Date, Category, Amount, Description
- Alternating row background colors (white and light blue-grey)
- Blue header row with white text
- Bold total row at the bottom
- Gracefully handles missing reportlab with an install instruction message

6. Data Storage

expenses.json - Sample Record

```
json
{
  "date": "2026-04-01",
  "category": "Food",
  "amount": 28.5,
  "description": "Groceries at supermarket"
}
```

Each expense is stored as a dictionary with four fields. The file holds a JSON array of all such records. Records are written after every add, update, or delete operation to ensure no data loss.

categories.json - Sample

```
["Food", "Travel", "Bills", "Entertainment", "Shopping", "Health", "Education", "Other"]
```

A flat JSON array of category name strings. Updated whenever categories are added, renamed, or deleted.

7. Menu System

The main() function drives the CLI with a continuous while True loop and a numbered input menu. The full menu structure is:

Option	Action
1	Add a new expense
2	View all expenses
3	View expenses by month
4	Update an existing expense
5	Delete an expense
6	Search by category
7	Search by amount range
8	Monthly text summary
9	Pie & bar chart visualization
10	Daily spending trend chart
11	Export to CSV
12	Export to PDF
13	View categories
14	Add a new category
15	Rename a category
16	Delete a category
0	Exit

8. Input Validation

Field	Validation Rule
Date	Must match YYYY-MM-DD; validated with <code>datetime.strptime</code>
Category	Must exist in the current categories list
Amount	Must be a valid float greater than zero
Description	Optional; defaults to "No description"
Menu choice	Handled with try/except <code>ValueError</code> for numeric inputs
Category name	Non-empty string; duplicates rejected; auto title-cased

9. Conclusion

The Smart Expense Tracker is a well-structured, object-oriented Python CLI application that covers the complete lifecycle of personal expense management. It demonstrates practical use of file I/O, data validation, analytical computation, data visualization, and report generation all within a single, self-contained class. The application is designed to be robust (graceful error handling, persistent storage, safe deletion), user-friendly (clear menus, helpful prompts, informative output), and extensible (custom categories, pluggable export formats). It serves as a strong demonstration of Python fundamentals applied to a real-world use case.